

Day-31 / RS 1 8/9/26

LC-2 $\rightarrow$ Add two numbers.

You are given two linked lists. Each linked list represents a number, but there is one imp. twist,

The digits are stored in reverse order.

Ex $l1 = [2 \rightarrow 4 \rightarrow 3]$
 $l2 = [5 \rightarrow 6 \rightarrow 4]$

O/p 342
 465

Now add them $342 + 465 = 807$
answer: $[7 \rightarrow 0 \rightarrow 8]$

$\rightarrow$ Why is the list stored in reverse?

$\rightarrow$ This makes addition ~~far~~ easier.

Normally, when we add numbers, we start from the rightmost digit:

$$\begin{array}{r} 342 \\ + 465 \\ \hline 807 \end{array}$$

$\rightarrow 2+5$ is calculated 1st.

But on a ll, we can only naturally move forward.
So, if the list is:

$2 \rightarrow 4 \rightarrow 3$

the 1st node is already the ones digit.
therefore, we reverse and start adding.

~~ex~~ $l1 = 2$

Approach - 1 | Brute force

1. traverse the 1st ll
2. convert it into the actual number.
3. traverse the 2nd ll
4. convert it into the actual number.
5. add the two numbers.
6. convert the answer back into a ll.

ex $l1 = [2, 4, 3]$ represents 2342.

$l2 = [5, 6, 4]$ represents 465.

then $2342 + 465 = 2807$.

finally: $[7, 0, 8]$

but there is a problem, the constraints allow up to:
100 nodes.

so, the number could contain 100 digits.
that won't fit inside:

int
long
long long.

therefore, for a constraint-safe bf soln in c++, we can
use strings to represent the nos.

string getNumber(ListNode* head) } this function converts the ll into a string representing the actual number.

string s = ""; } creates an ~~empty~~ empty string.

while (head != nullptr) } traverse the entire ll.

{
s += char(head->val + '0'); } convert the digit into a character and add it to the string.
head = head->next;
}

char(-) explicitly converts the Integer into a char.

ex
head->val = 2,
then
2 + '0'
= 50

char(s)
= '2'

So, it convert the digit stored as an int. into its corresponding char.

and s += adds

one after the another.

"2" + "24" -> "243"

head->val = 2
head->val = 4 then
head->val = 3.

our ll contains int val;

So,

head->val is an int.

but s = char.

so we did s +=
and also, integer 2 -> char '2'.
↓

2 + '0'
↓
'2'

So, '0' is added.

Why does this work?

→ both char have memory code (ASCII)

for ex, '0' = 48
'1' = 49

So, 2 + '0' = 2 + 48 = 50
and ascii value 50 represents: '2'

reverse(s.begin(), s.end());
return s;

↓

reverse it,

→ give me an iterator pointing to the 1st char of s, and give me an iterator pointing just after the last char, and

reverse all chars from the beginning of s upto the end of s.

Suppose, s = "243",
but, the linked list stores digits in reverse order.

* but normally we use array, 342

Reverse = 243.

and that's how they are stored in a list.

243

2 → 4 → 3
↓ ↓ ↓
ones tens hundreds.

actual = 342.

string addStrings (string a, string b) {
string result = "";
→ creating empty str to store our answer.

int i = a.size() - 1; → for a = "342"

int j = b.size() - 1;

int carry = 0;

→ this stores the carry from add.

initially, no carry,

so, carry = 0,

but for 9+8=17,

no store 7,

and carry 1:

int i = 2

i points to 342

why do we start from the end? → but normal addition starts from the rightmost digit.

→ function receiving 2 strings.

why strings instead of int? → but the linked list can contain upto 100 digits.

→ a c++ cannot hold a 100-digit number.

$\rightarrow$ for $j = -1 \rightarrow$ nothing left
 even if both numbers are finished, there can be a carry that needs to be added.
 while $(i \geq 0 \parallel j \geq 0 \parallel \text{carry})$ { keep doing the add'n as long as there is something left to process.

int sum = carry;
 if $(i \geq 0)$
 $\text{sum} += a[i--] - '0';$
 if $(j \geq 0)$
 $\text{sum} += b[j--] - '0';$

we start sum with whatever carry comes from the prev add'n.

if carry = 1.
 so, the next iteration starts with, sum = 1.
 then we add the two digits to it.

~~no sum += ch~~
 $i = 2$.
 $a = 242$
 $a[i] = a[2] = '2'$, post decrement then $i = i - 1; i = 2, i = 1$.

~~for $i = 0$~~
 initially,

~~$a[i--]$~~
 $a = "242"$
 $b = "465"$
 $i = 2$
 $j = 2$
 $\text{carry} = 0$.

$i = 2$
 $j = 2$
 $\text{carry} = 0$.

Hand 1
 $\text{sum} = 0$.
 $i = 2, a[2] = 2$.
 take digit from b.
 $j = 2$, so,
 $b[2] = 5$.
 therefore, $\text{sum} = 2 + 5$
 then $i = 1, j = 1$.

Step 1
 $\text{sum} = \text{carry}$;
 $\text{sum} = 0$.
 ~~$\text{sum} += a[i--] - '0';$~~
 $i = 2$ so,
 $a[2] = 2$

take digit from b,
 $j = 2$, so, $b[2] = 5$.
 $\text{sum} = 2 + 5 = 7$.
 then $i = 1, j = 1$, $\text{sum} = 7$, $\text{carry} = 0$.

Recursion 2

sum = 0.
a[1] = 4
sum = 0 + 4 = 4.
then i = 0,

hence $b[i] = b[1] = 6$.
So,

sum = 4 + 6 = 10
then j = 0.

Result += char ((sum % 10) + '0');
carry = sum / 10;
↓
carry

gives us the last digit. we store
the digit that belongs
on the current position
and put it in to Result.

Ex 1st hand
2 + 5 = 7.
sum = 7.

digit:
7 % 10 = 7
add to Result:-
Result = "7"

carry = 7 / 10 = 0
So carry = 0

2nd hand
4 + 6 + 0 = 10
sum = 10
digit:- 10 % 10 = 0.

sum = 10.
but, we can't put 10 into
one digit, posch.
no need.

10 → 0 1
↓ ↓
if sum digit is store → carry.

add:-
Result = "70"
carry:- 10 / 10 = 1.
carry = 1.

2nd hand
3 + 4 + 1 = 8
sum = 8.
digit = 8 % 10 = 8, add,
Result = "708".
carry = 8 / 10 = 0


```

ListNode* buildList (string s) {
    ListNode* dummy = new ListNode(0);
    ListNode* current = dummy;
    for (char c : s) {
        current->next = new ListNode(c - '0');
        current = current->next;
    }
    return dummy->next;
}

```

creates a temporary / helper node.
 dummy & current both point to the same helper node.
 current will move forward as we create new nodes.
 take (char c : s) → take every char of the string one by one.
 create a node containing (c - '0') and after connect it to current.
 current → next = new ListNode(c - '0');
 current = current → next;
 move current, actually immediately after attaching current
 is still at dummy, so move it.
 returns head of the list.

```

ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
    string num1 = getNumber(l1);
    string num2 = getNumber(l2);
}

```

```

string sum = addStrings(num1, num2);
reverse(sum.begin(), sum.end());
return buildList(sum);
}

```

addTwoNumbers() is the main function, and buildList() is a helper function used at the very end to convert our answer from a string back into the ll format that it wants.

we get "807"

but it wants

we got "708" but we want 708 → 807 ll.
 that's exactly why buildList() exists.

2. ~~1.1~~

ListNode * buildList (string s) $\rightarrow$ i have a string s, and i want to build a ll from it.

~~f.c = $O(n)$~~ $\rightarrow$ no. of nodes in ll
~~Line of nodes in ll.~~

~~s.c = $O(1)$~~

f.c = $O(n+m)$ $\rightarrow$ no. of nodes in ll
 $\rightarrow$ no. of nodes in ll.

s.c = $O(n+m)$ but n1 string $\rightarrow O(1)$
n2 string $\rightarrow O(m)$

Why its not optimal & although the asymptotic complexity is still $O(n+m)$, we're doing unnecessary conversions:-

ll
 $\downarrow$
string
 $\downarrow$
string addition
 $\downarrow$
string
 $\downarrow$
ll.

$\therefore$ we can simply add the ll digits directly. That's why the next approach is cleaner and preferred.

Approach - 2 / optimal

`ListNode* dummy = new ListNode(0);` creates a dummy node
~~`int carry = 0;`~~ matches constructing the answer much easier.

`ListNode* current = dummy;` `current` points to the position
`int carry = 0;` where we'll insert the next
result node.

↳ this stores the carry from the previous addition.

`while (l1 != null || l2 != null || carry != 0) {` continues
while there are

`int sum = carry;`

`if (l1 != null) {`

`sum += l1->val;`

`l1 = l1->next;`

}

single digits in `l1`, `l2`,
and `carry` that
needs to be added.

suppose, `l1 = 2 → 4 → 3`

`l2 = 5 → 6 → 4.`

Remember: the `ll` is already in reverse
order, so we can directly add
corresponding nodes.

→ we start `sum` with whatever `carry` came
from the previous addition.

for 2 and 5, no carry, so
`sum = 0.`

`l1->val =` gives the value stored in the node
pointed to by `l1`.

`sum = sum + 2`

`= 0 + 2 = 2.`

then `l2` adds its digit, `l2->val = 5.`

`sum = 2 + 5`

`= 7.`


```

if (l2 != null ptr) {
    sum += l2->val;
    l2 = l2->next;
}

```

create a new node,
 $(sum / 10)$ used to get the
 left digit of sum -
 the digit that
 should be
 placed in the

```

current->next = new ListNode (sum % 10);

```

current
 linked list

carry = sum / 10; → calculate carry for
 the next addition

```

current = current->next;

```

```

return dummy->next;

```

```

}

```

for ex, sum = 10,

we get
 $10 / 10 = 1$

carry = 1.

but,

$10 = 1 \text{ ten} + 0 \text{ ones}$.

the 0 goes into the current
 node, while the 1 is
 carried to the next
 column.

this puts current
 to the node we just created.

time complexity: $O(\max(n, m))$
 ↳ no. of nodes in list
 ↳ no. of nodes with.

space complexity: $O(1)$
 ↳ as we traverse each node
 only once.

↳ both use pointers the longer
 list
 completes.

time complexity: $O(\max(n, m))$